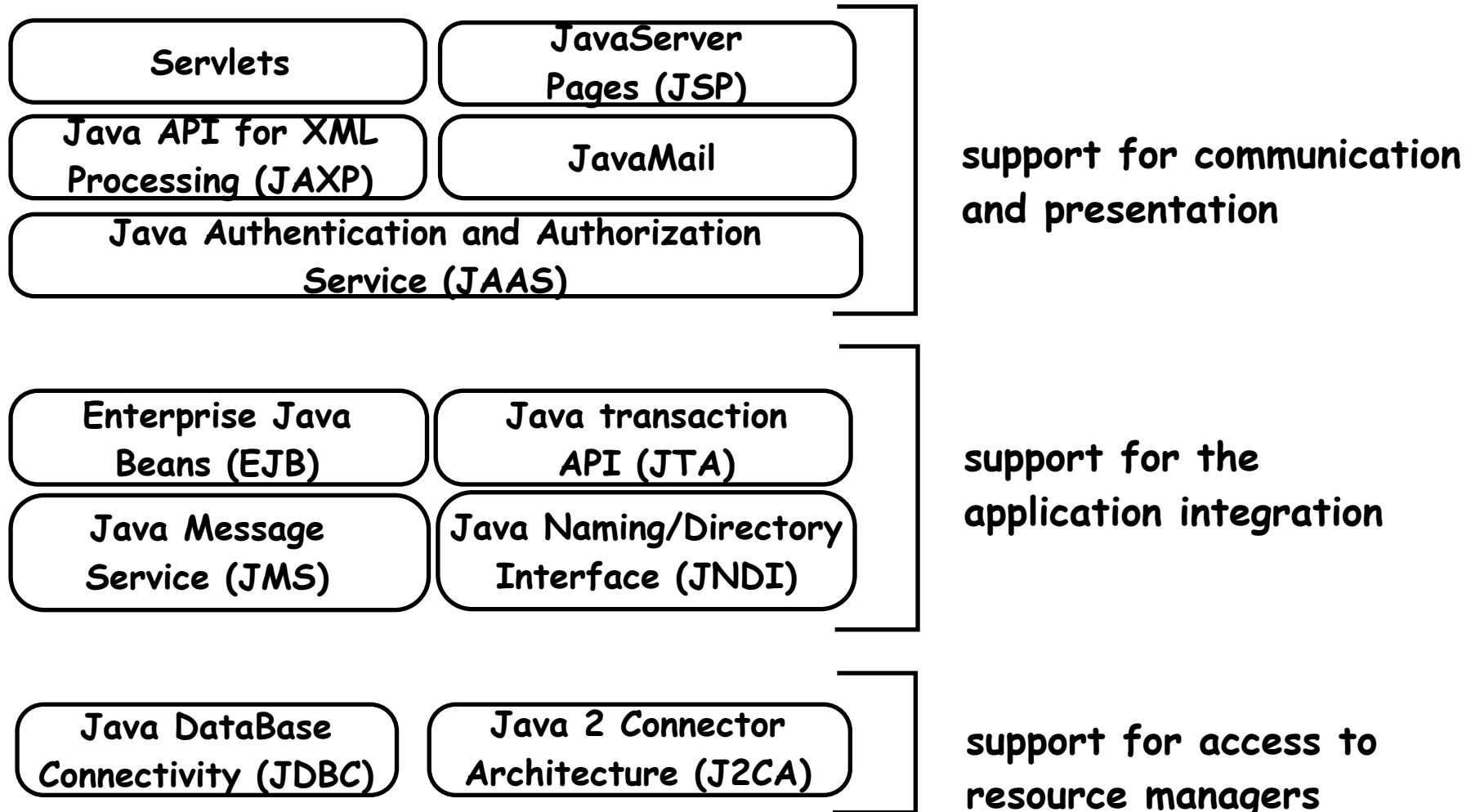
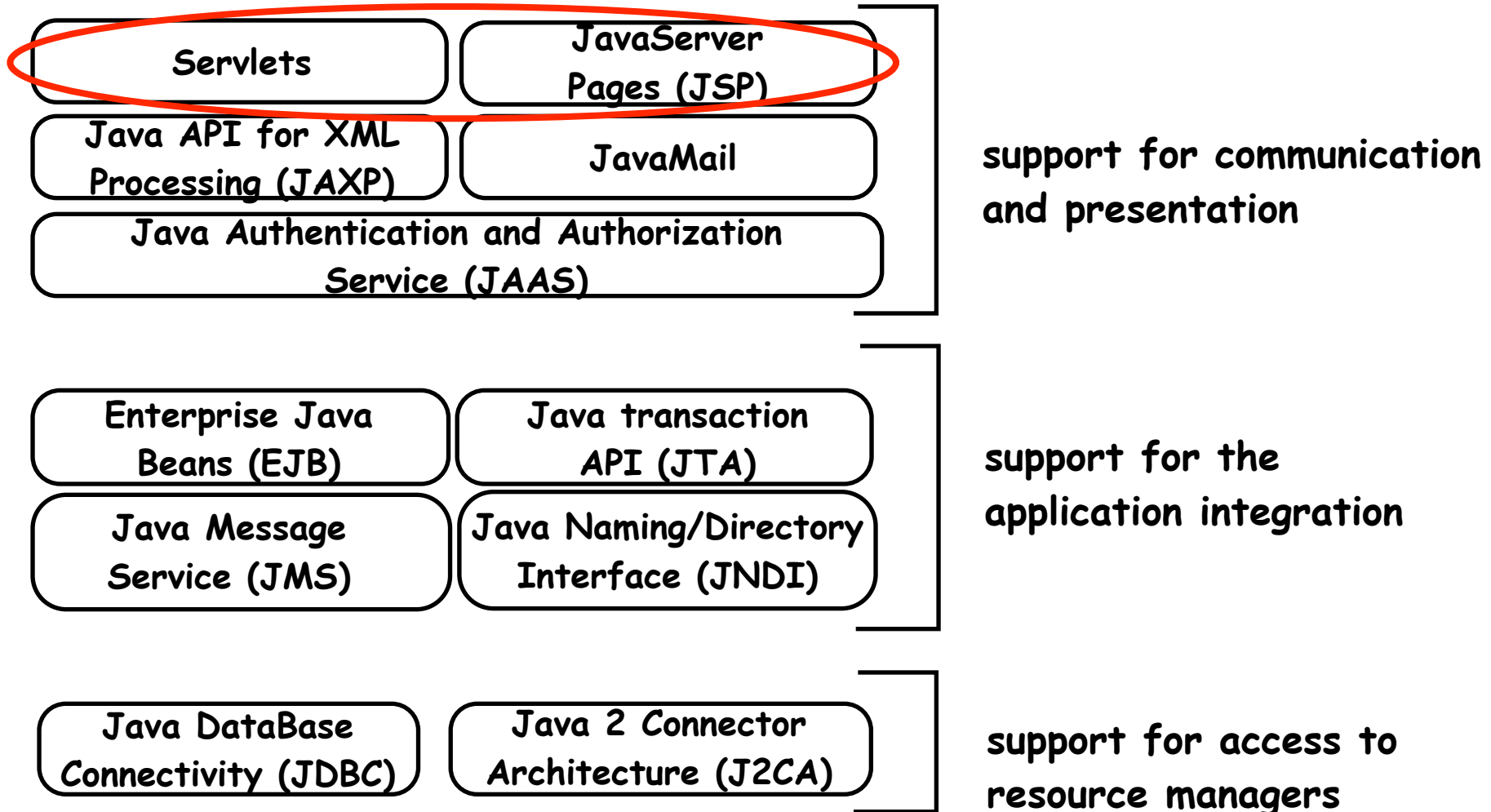


- Portable Standard
- Implements (more-or-less) most of the stuff we have described so far
- [ACKM04] Ch 4
- <http://java.sun.com> aka www.javasoft.com

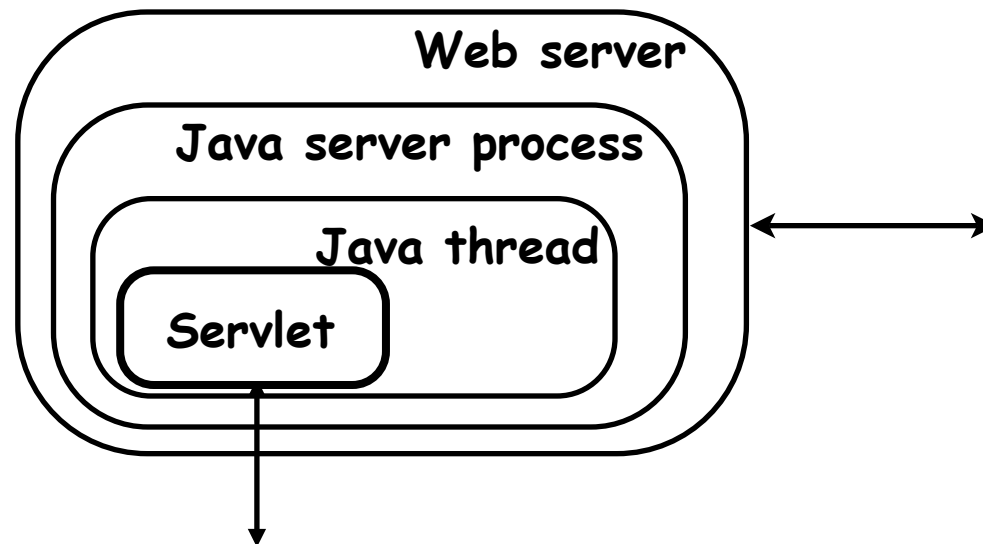
J2EE Application Server Architecture



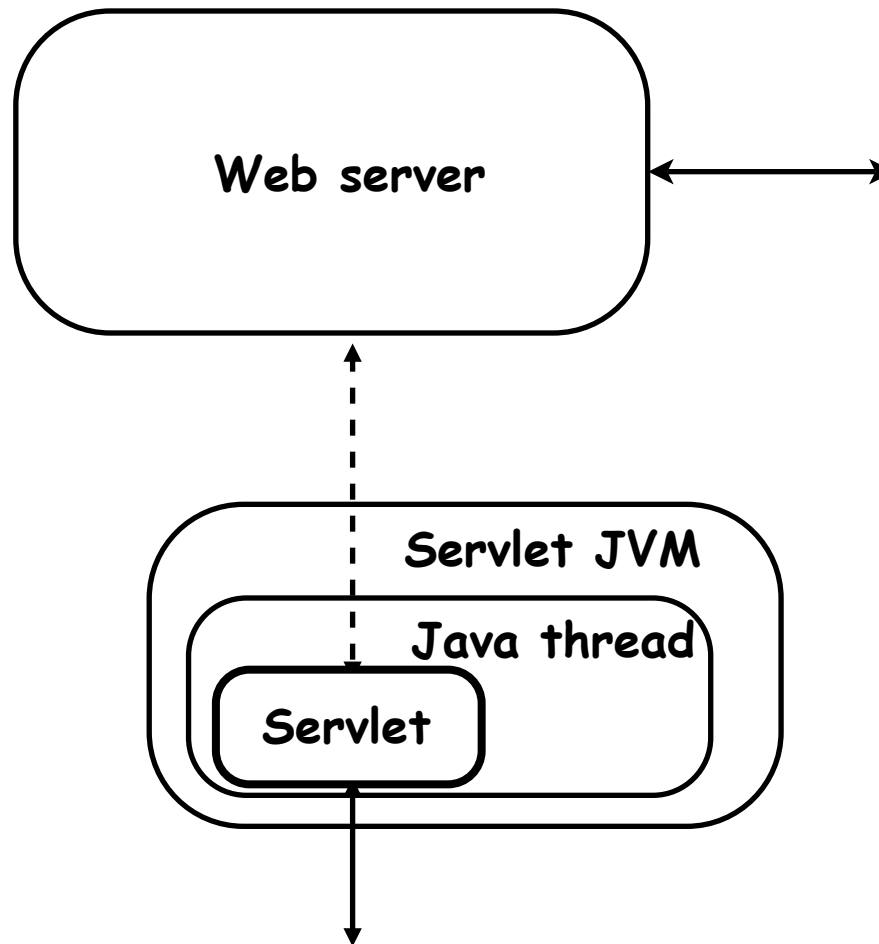
J2EE Application Server Architecture



Servlets

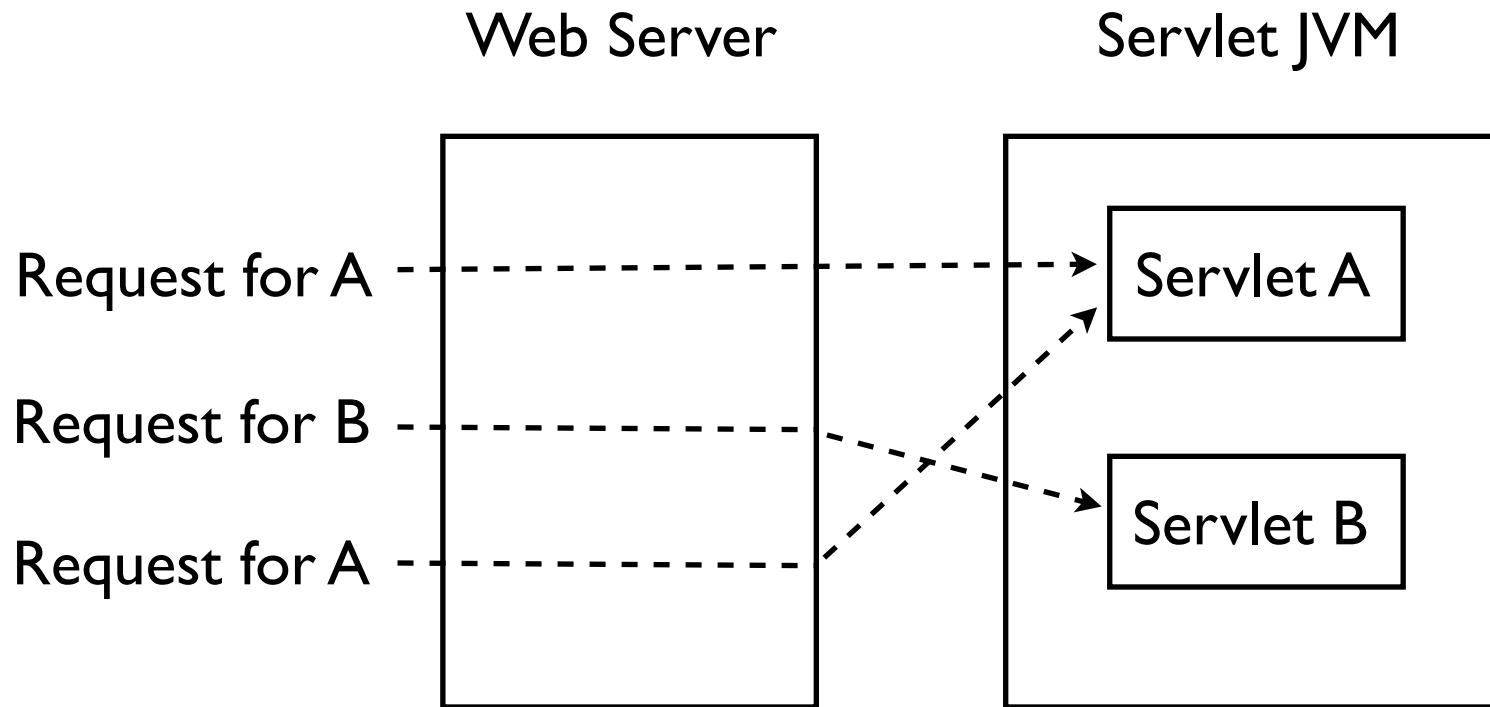


Servlets



- JVM can be separate from Web Server
- Still avoids process creation overhead

Servlet Life Cycle



- First reference creates servlet object
- Subsequent references just invoke methods
 - request and response parameters

Servlet Life Cycle

- Servlet notified when created and destroyed
- Can save state between calls

```
public class HitCountServlet extends HttpServlet {  
    int timesAccessed;  
    public void init(ServletConfig conf) ... {  
        timesAccessed = 0;  
    }  
    public void doGet(  
        HttpServletRequest req, HttpServletResponse resp) ... {  
        ... timesAccessed++; ...  
    }  
}
```

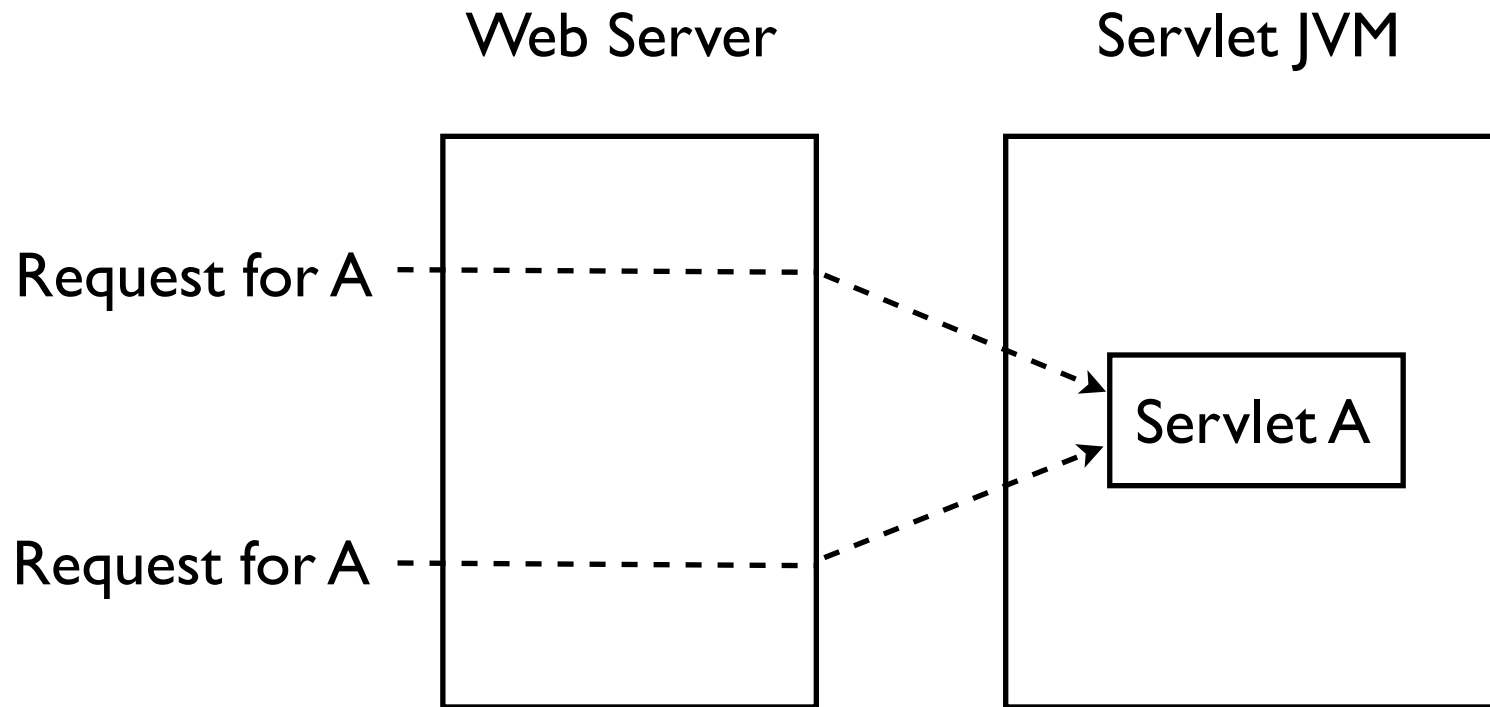
It is more complicated than that.
Servlet may be destroyed by container anytime,
it must save state to stable storage:

```
public class HitCountServlet extends HttpServlet {
    ...
    public void destroy() {
        ...
        outfile.writeInt(timesAccessed);
        ...
    }
}
```

Servlet Life Cycle

- Real examples: serially reusable resources that are expensive to create
 - Database connection pools

Concurrency - Thread Safety

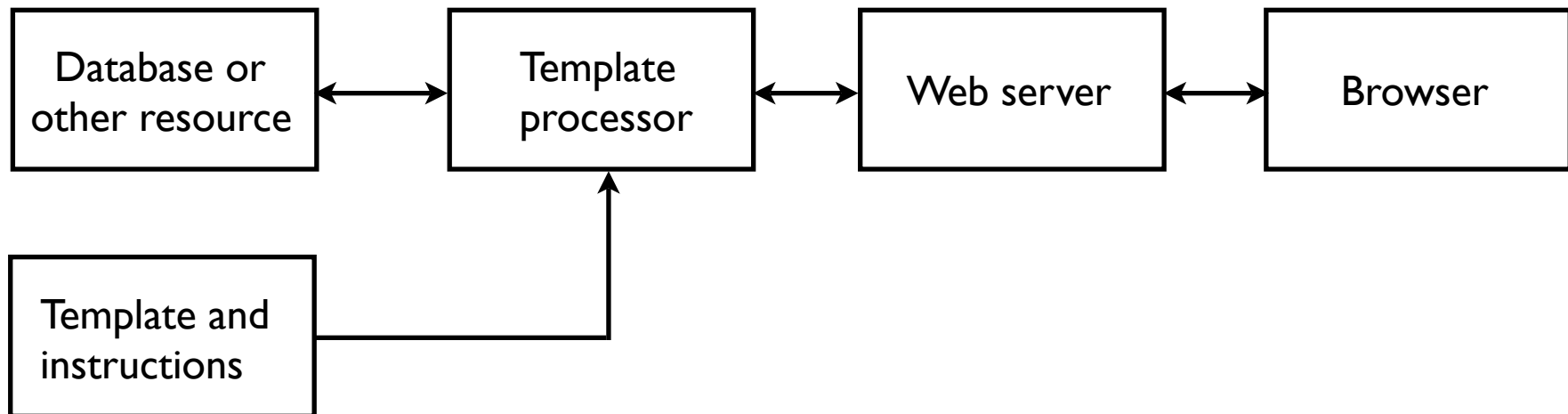


- Each call has its own req and resp parameter
- But servlet code -- *and all code called by the servlet* -- must be thread-safe!

```
...  
HTTPSession theSession = req.getSession(true);  
...
```

- Server will create session context
 - assign unique ID
 - track session by cookies or URL encoding or ... (not part of spec)
- Servlet can store (anything) in session context, it will persist between calls
 - even if servlet is destroyed/recreated

- *A template engine:*



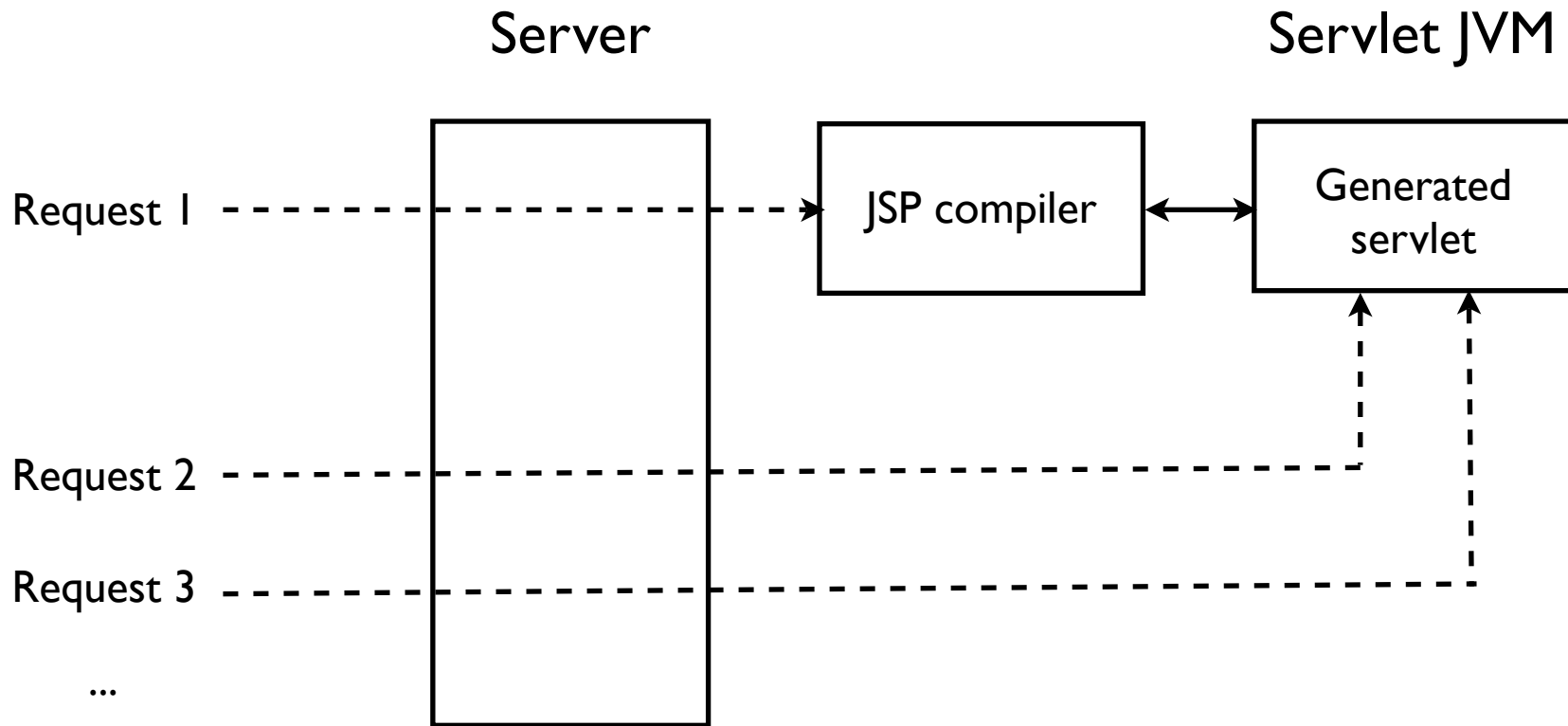
- A JSP template is HTML with snippets of Java embedded in it
- Here is a really simple one ...

```
<html>
<body>
  Hello, visitor. It is now
    <%= new java.util.Date().toString() %>
</body>
</html>
```

- Here is one with control flow!

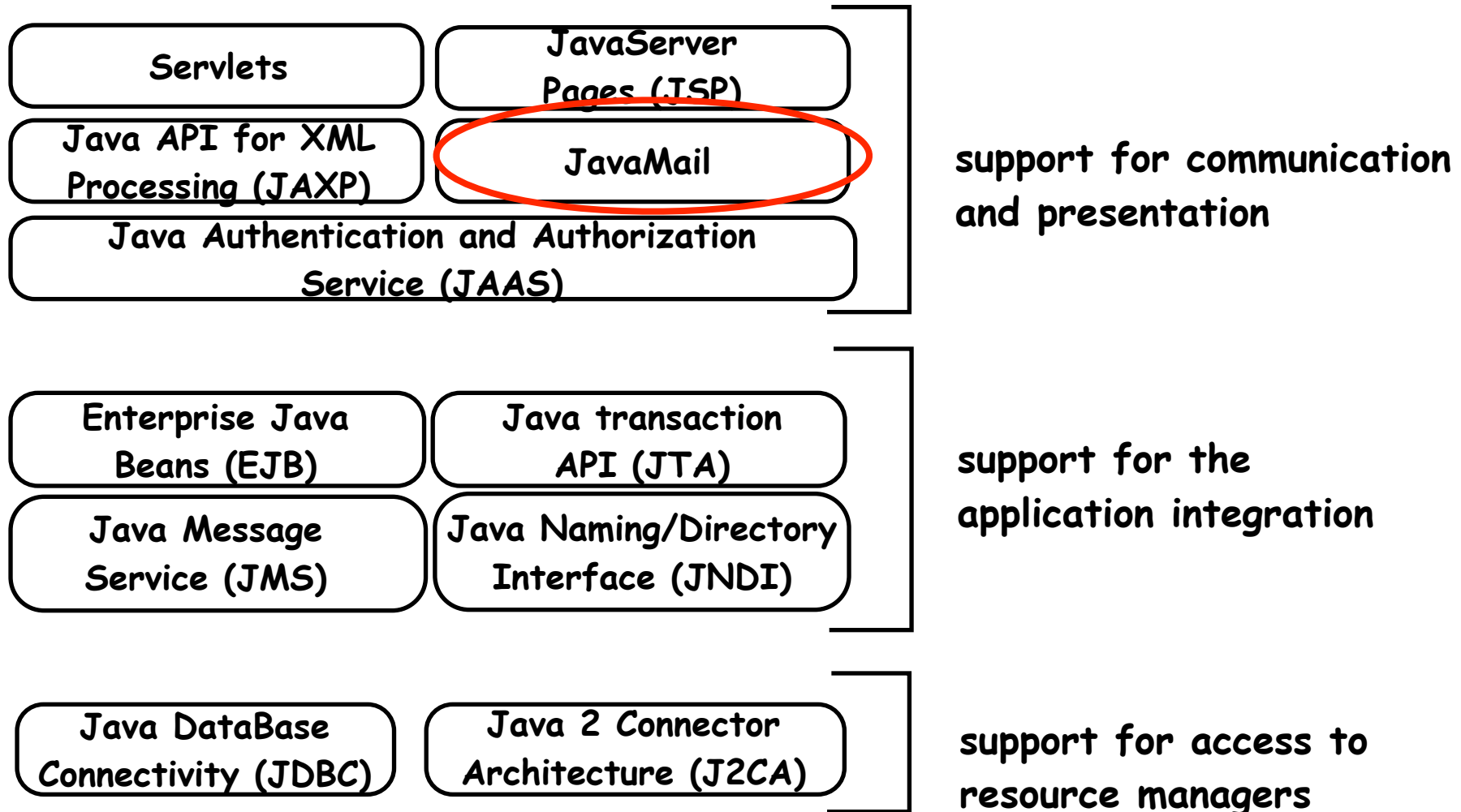
```
<html>
<body>
<% java.util.Date theDate = new java.util.Date(); %>
<% if (theDate.getHours() < 12) { %>
Good morning,
<% } else { %>
Good afternoon,
<% } %>
visitor. It is now <%= theDate.toString() %>
</body>
</html>
```

JSP Life Cycle



- The template is compiled (once) into a servlet
- Later references re-use existing servlet

J2EE Application Server Architecture



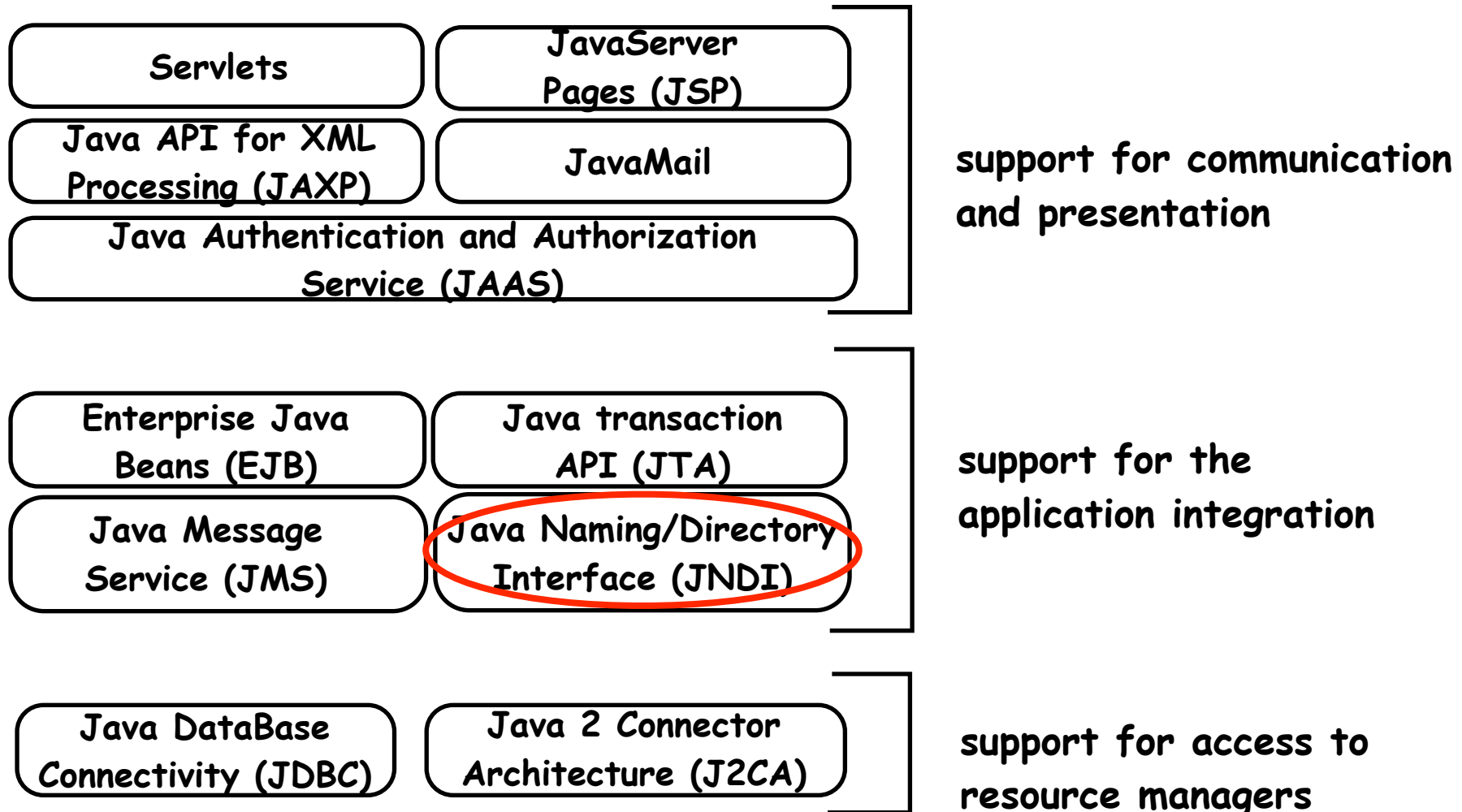


JavaMail

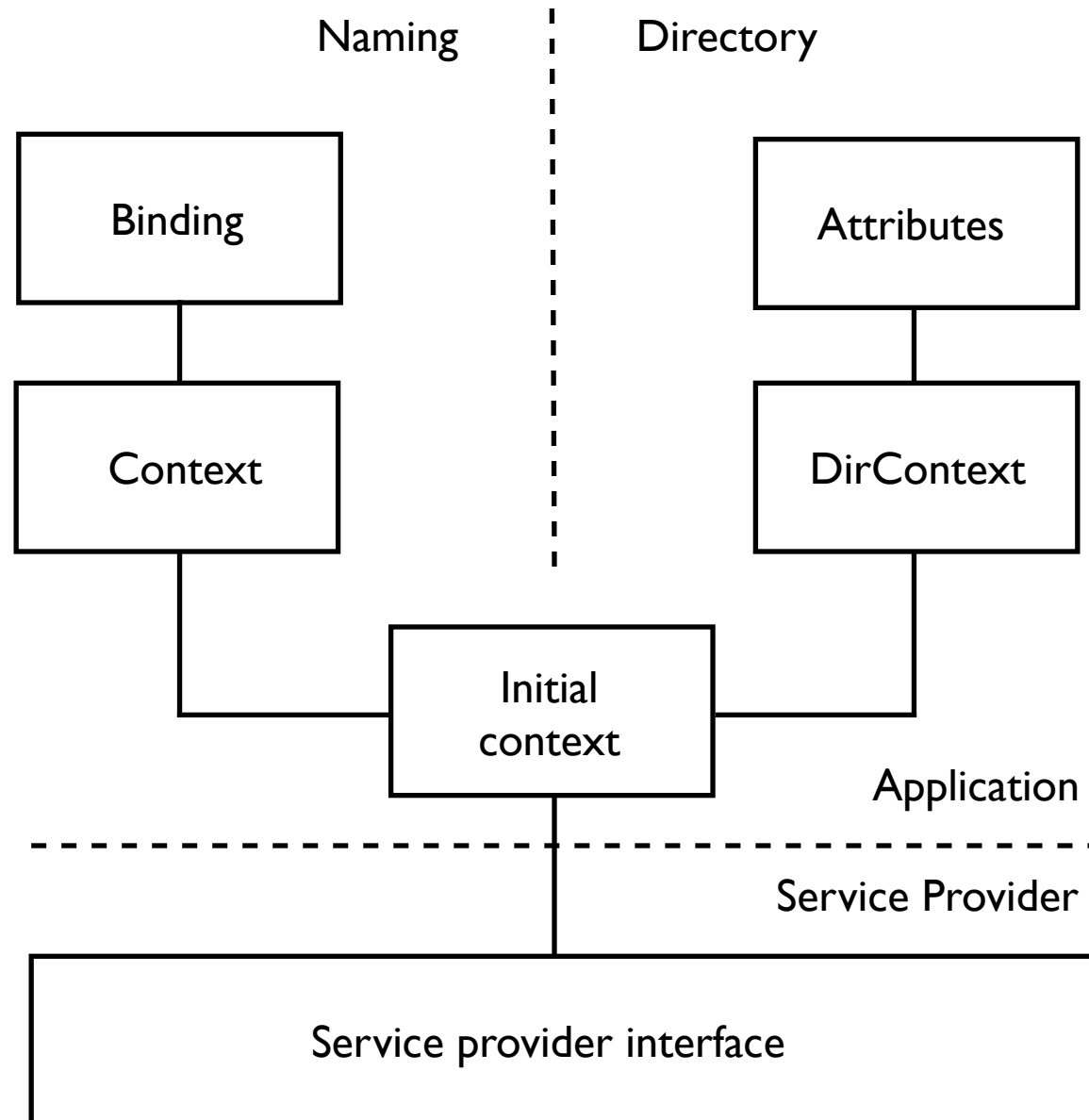


- Send and receive mail objects
- Straightforward

J2EE Application Server Architecture



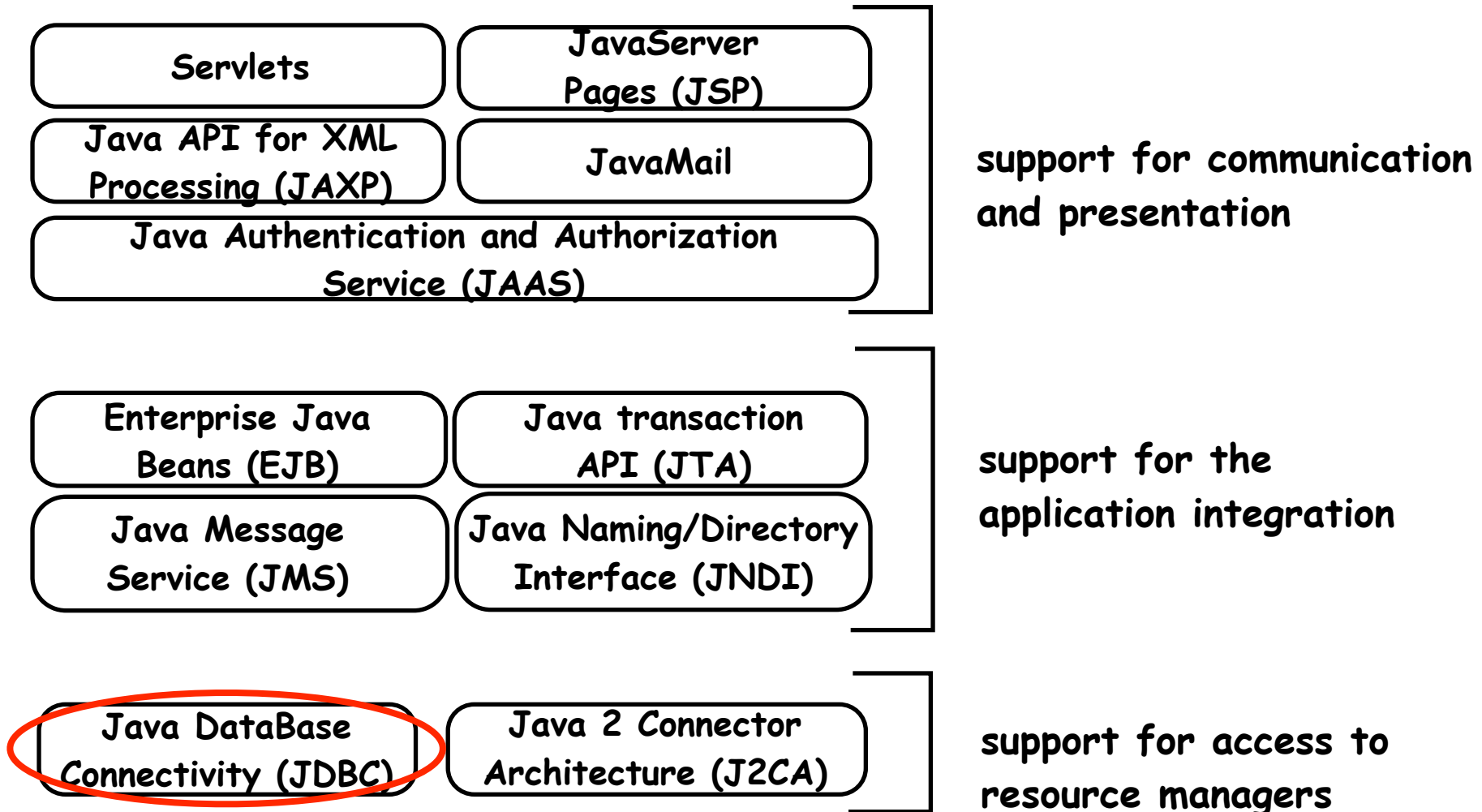
JNDI Architecture



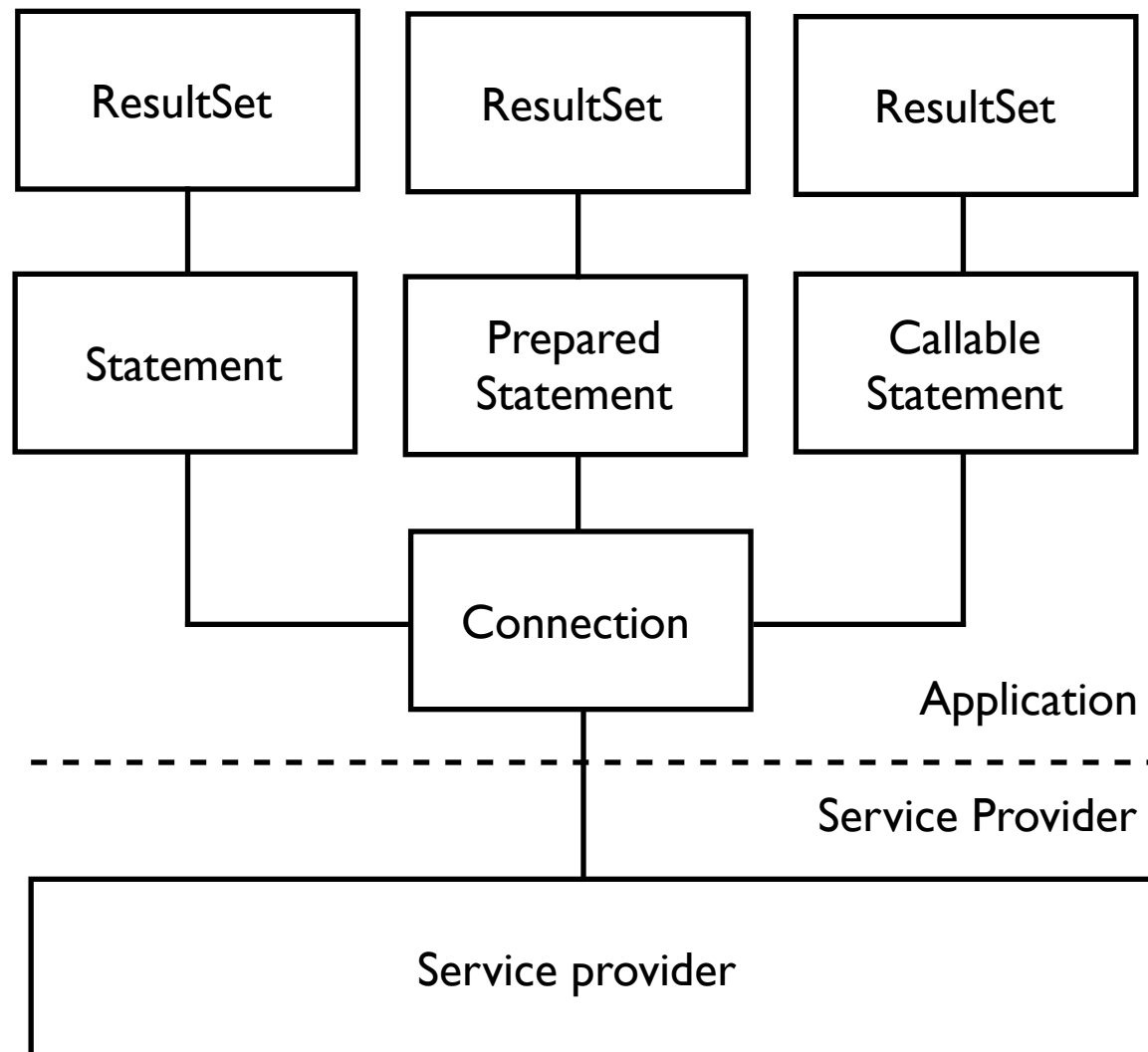
Naming and Directory

- Naming service
 - context = bindings of names to objects or contexts
 - enumerate subcontexts
- Directory service
 - dircontext = bindings of names to objects with attributes or dircontexts
 - enumerate attributes
- Initial Context = where you start looking for objects / services
- (like X.500 or LDAP)

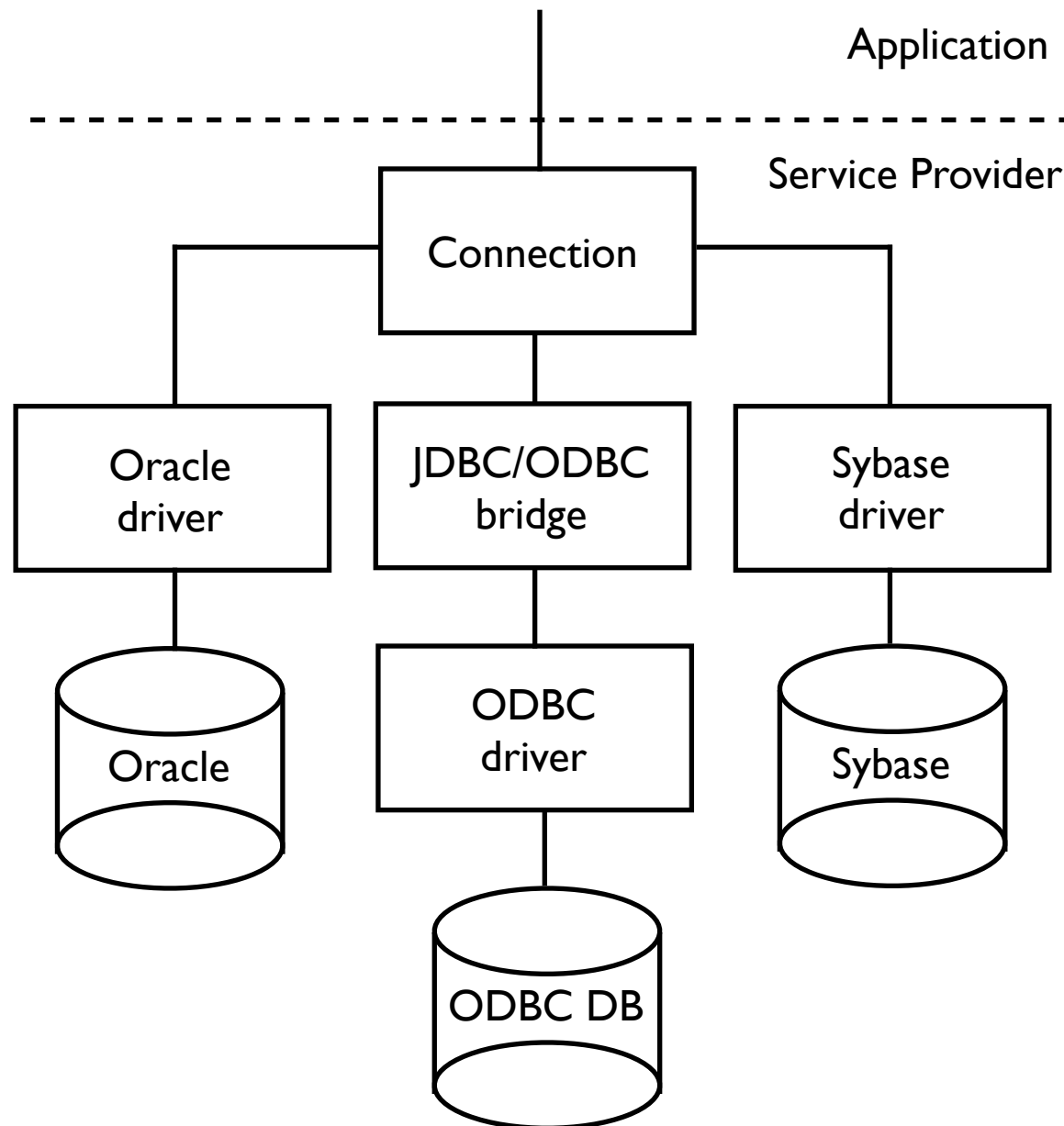
J2EE Application Server Architecture



JDBC Architecture



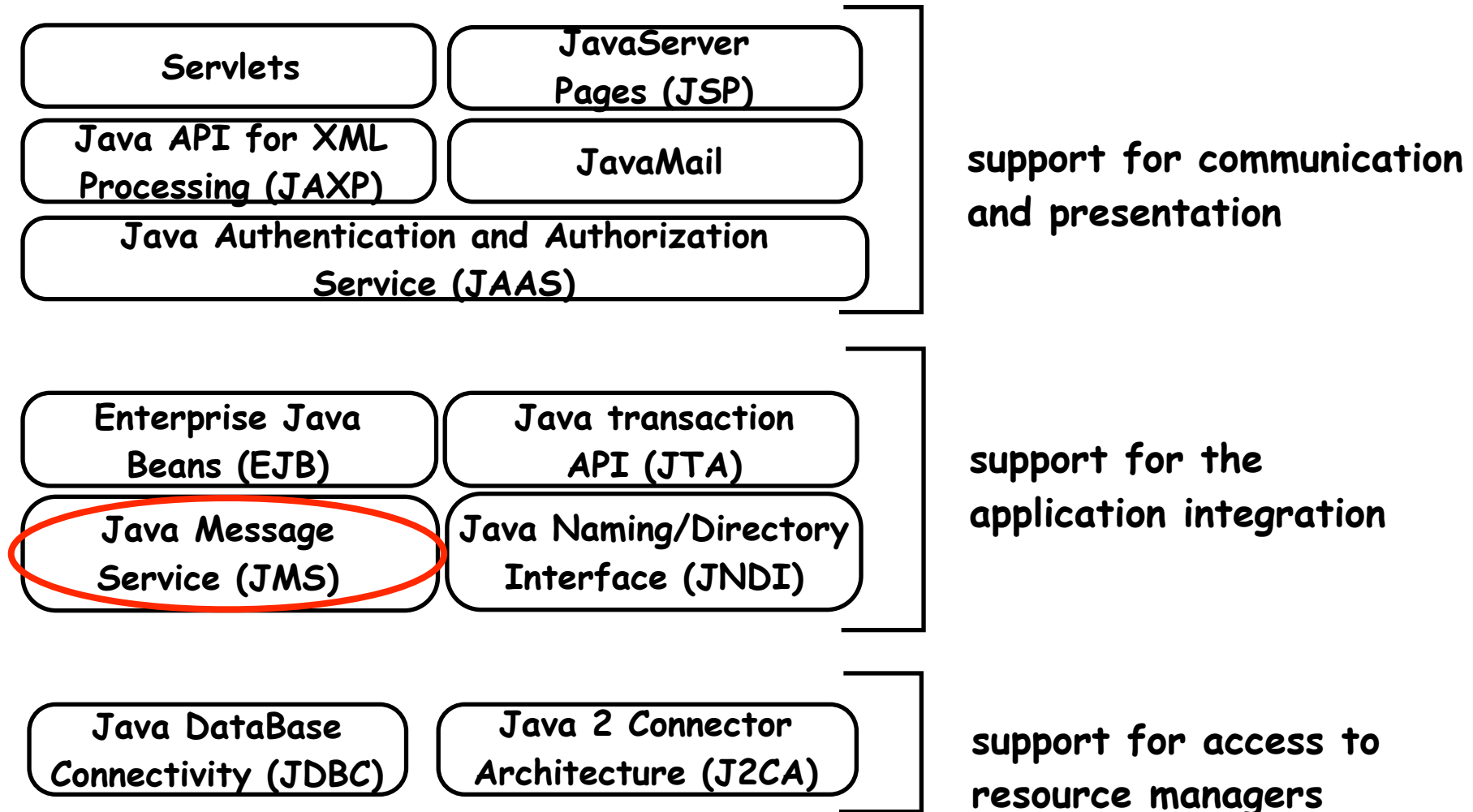
JDBC Architecture II



- Connections
 - pooling support
- SQL queries
 - precompiled queries
 - database stored queries
- Result sets
 - enumeration
 - scrollable

- Transaction control
 - auto-commit
 - explicit commit/rollback
 - savepoints
- Isolation levels
 - READ_(UN)COMMITTED
 - REPEATABLE_READ
 - SERIALIZABLE
- Distributed transactions
 - if supported by the database and driver

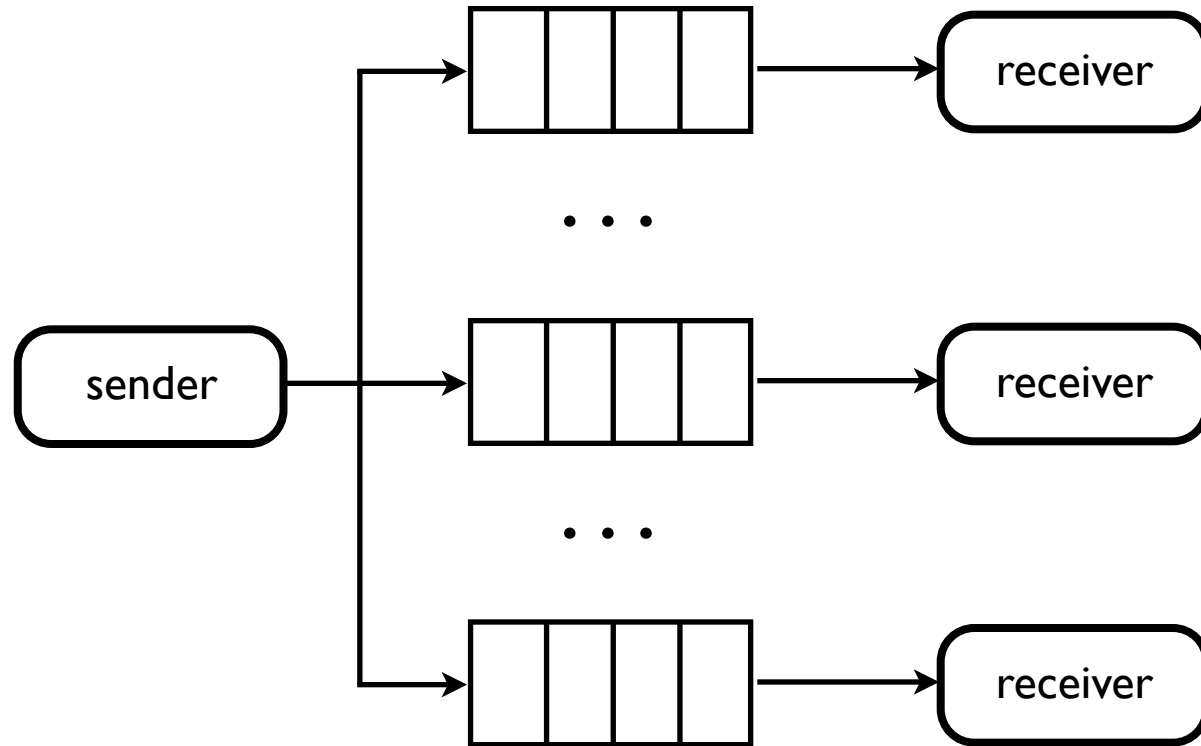
J2EE Application Server Architecture



Point-to-Point

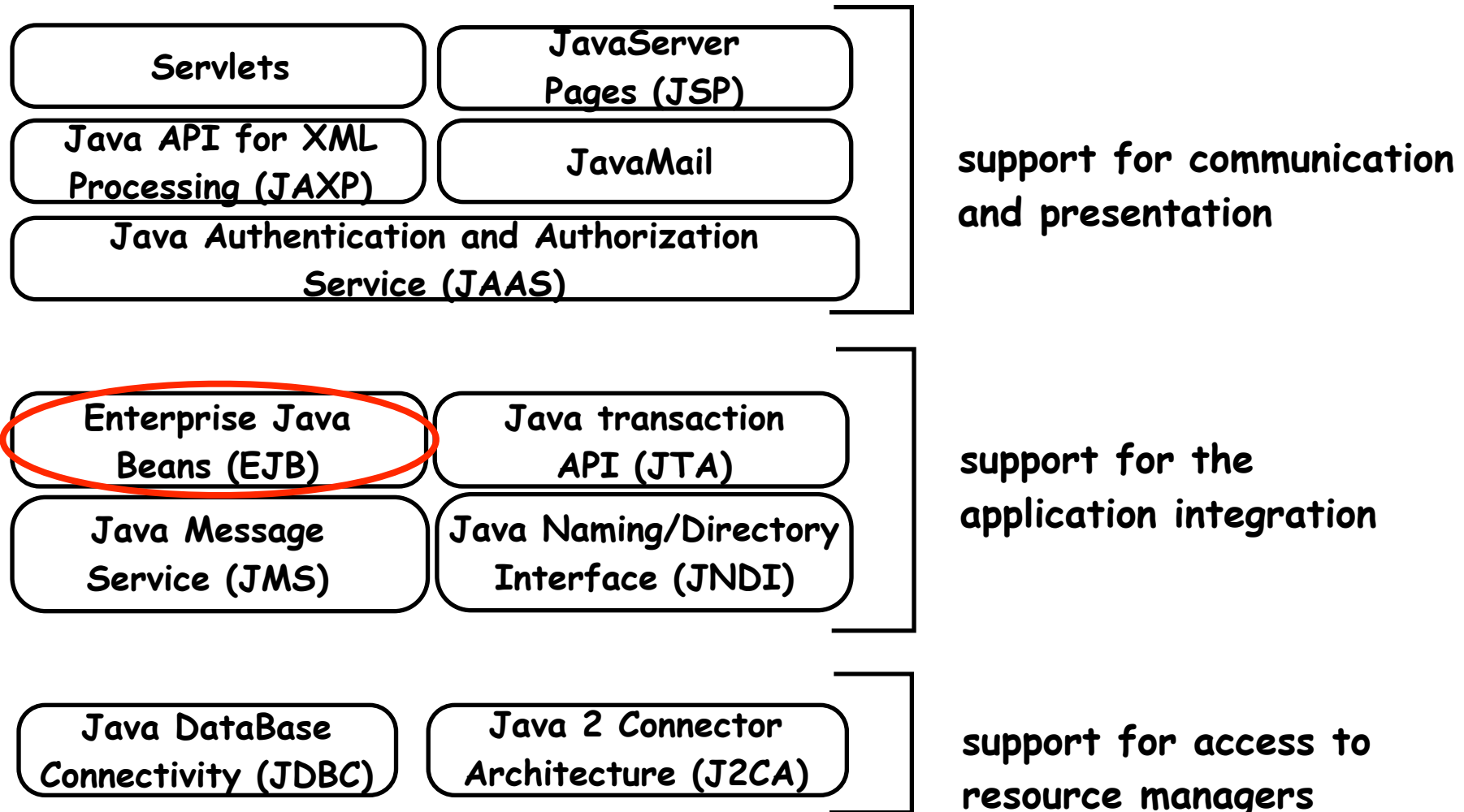


Topic Based (pub/sub)



- Message filtering (Boolean predicates) at receiver
- Durable subscriptions if desired
- Transactional support
 - JMS queues only
 - or other resource managers as well

J2EE Application Server Architecture



What is a bean?



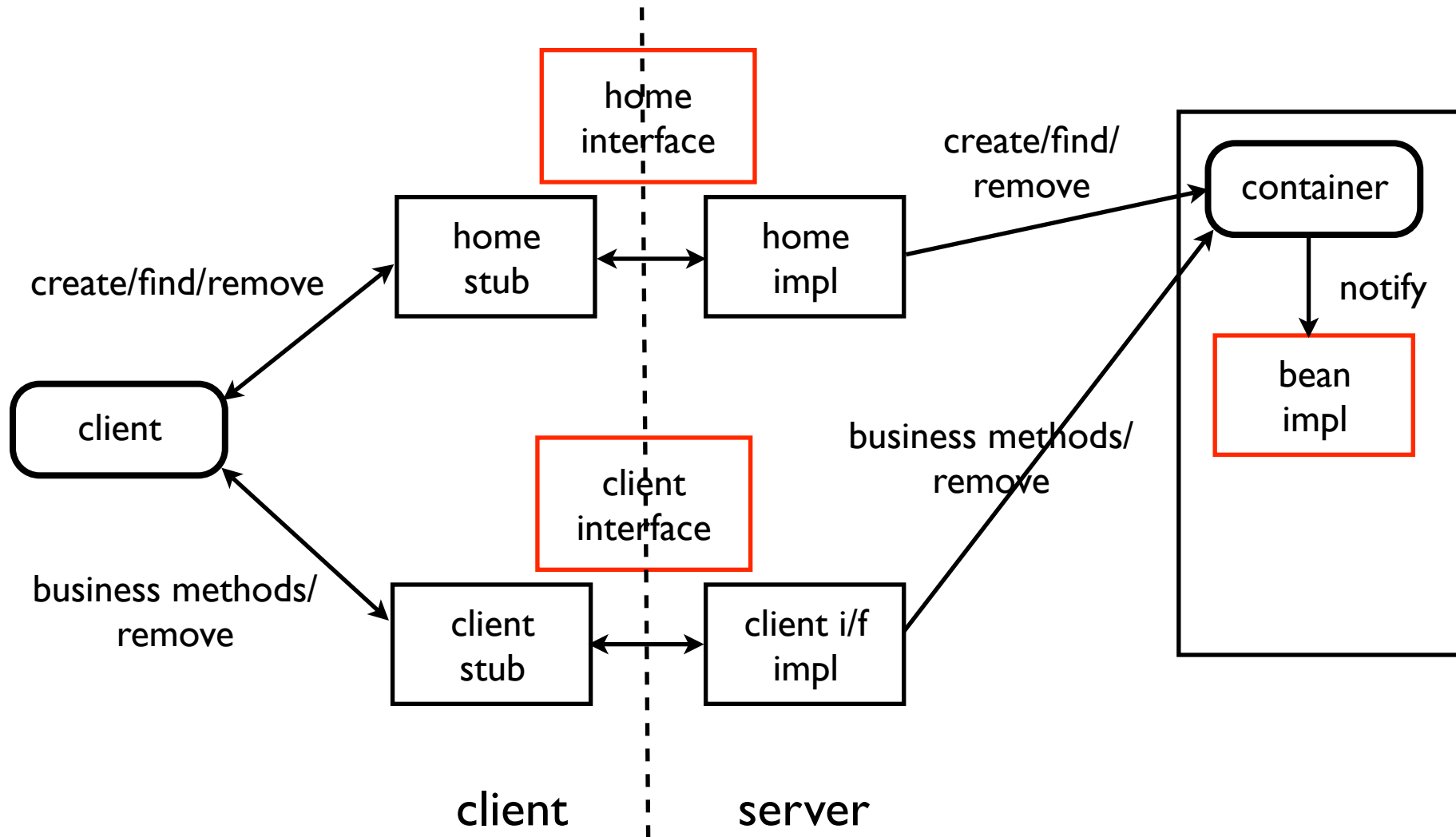
- Java Object
- Container provides many standard services
 - location / binding
 - life cycle
 - persistence
 - transactions
 - etc

Classes of Beans



- Session
 - embodies business logic associated with a session / business txn
- Entity
 - represents a real-world object
- Message-driven
 - asynchronous processing using JMS queues

EJB Classes and Stubs



Session Beans

- embodies business logic associated with a session's / business txn
- always given a session context
- not persistent
- stateful
 - invoked only thru creating session
- stateless
 - use by concurrent sessions allowed

Entity Beans

- represents a real-world entity
- may be shared by multiple sessions
- persistent beyond session or EJB container lifetime
- persistence may be *bean-managed* (JDBC/JTA) or *container-managed*

Bean-Managed Persistence

- Container notifies bean on activation/passivation
- Bean implementation responsible for serializing state to database
- Bean implementation responsible for transaction behavior (JTA)
 - distributed txns may be supported

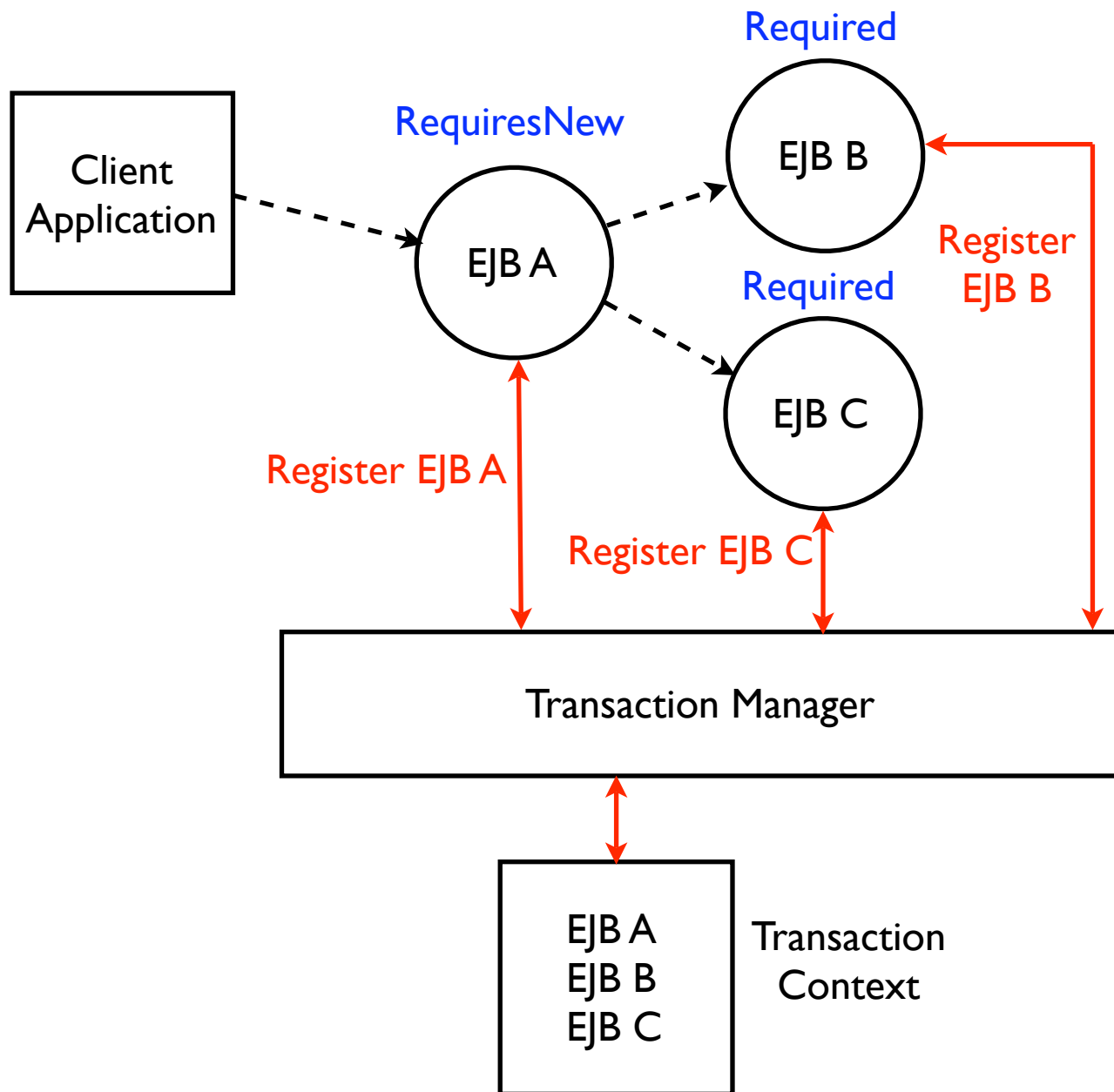
Container-Managed Persistence

- Container manages serializing state to database and retrieving
- Container manages transactional behavior
 - declarative

Declared Transaction Attributes

- NotSupported
- Supports
 - use txn if present
- Required
 - create txn if none
- RequiresNew
 - create txn
- Mandatory
 - error if no txn
- Never

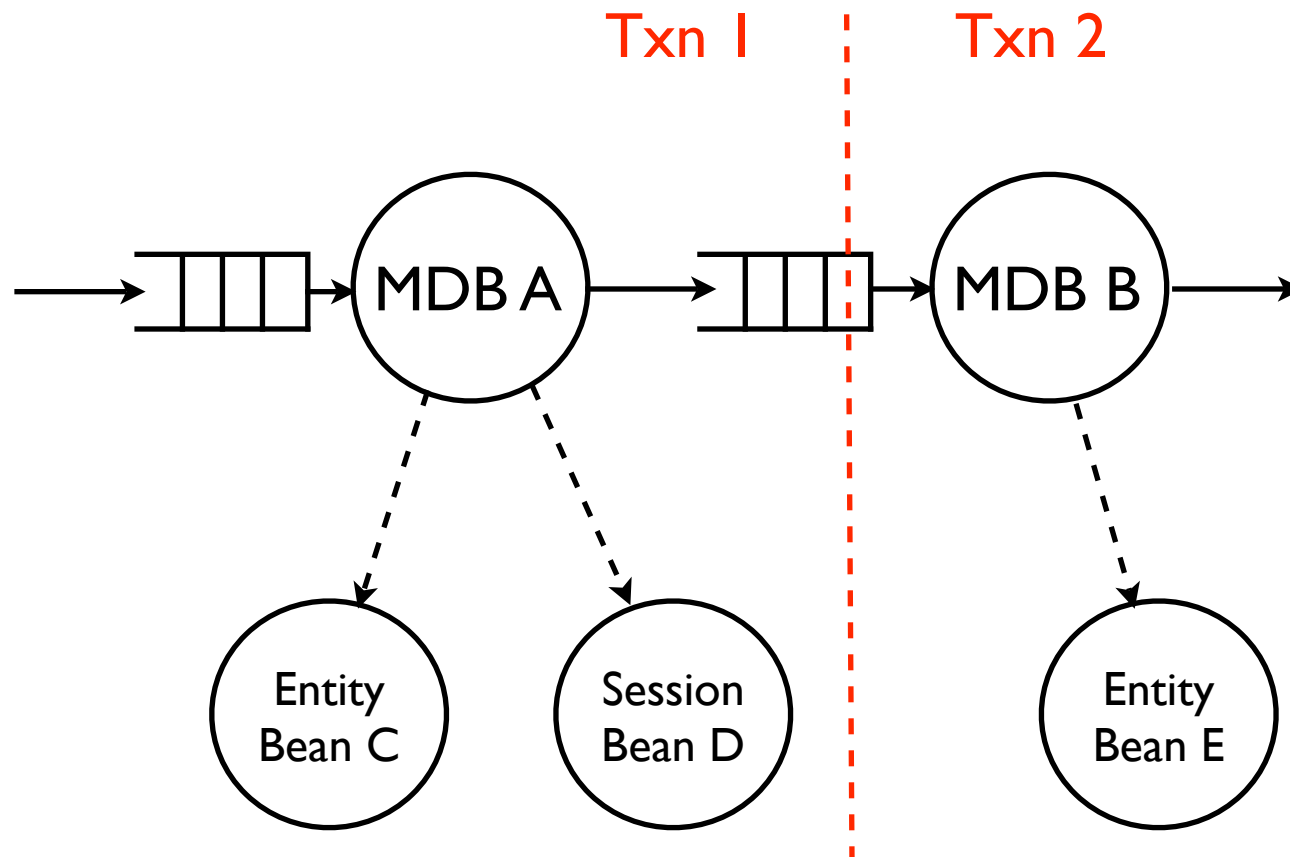
Managing a Transaction Context



Message-Driven Beans

- Communication by queueing (JMI) rather than RPC
- Container manages
 - creation of MDB when message arrives
 - invoke onMessage method with each arriving message
 - remove idle beans
- Equivalent to dequeue-process loop

Message Driven Beans



- XML deployment descriptors
 - identify programmer-provided code
 - dependencies
 - transactional behavior
 - security properties
- Container generates stubs